



Second Brain mit Claude Code

Die Bauanleitung

Von der zentralen Frage zum System, dem du vertrauen kannst — Schritt für Schritt, mit allen Vorlagen zum Kopieren.

DIE ZENTRALE FRAGE

Wie wird aus gesammeltem Wissen ein System, dem ich **vertrauen** kann — statt ein wachsender Müllhaufen?

Diese Frage hat zwei Hälften: **Auffindbarkeit** (jede Information wiederfinden, auch ohne das exakte Wort) und **Vertrauen** (Duplikate, Widersprüche und veraltete Stände sollen auffallen, statt still herumzuliegen). Mit der Graph-Optik hat das alles noch nichts zu tun — die Optik ist ein Fenster, und ein Fenster baut man zuletzt ein.

Bevor du ein Werkzeug anfasst: **Fähigkeiten zuerst**

Schreib auf, was dein System können muss, damit du ihm vertraust. Als Vorlage die vier Fähigkeiten aus meinem System — jede bekommt später genau ein Bauteil:

Fähigkeit	Bedeutet	Bauteil
Finden	Relevante Informationen in Sekunden — auch ohne das exakte Wort	qmd-Suche
Lesen	Treffer direkt öffnen, ohne App-Wechsel	Obsidian / Inspector
Sauber bleiben	Widersprüche erkennen, Wissen verdichten	Wiki-Schicht
Überblick	Das ganze System auf einen Blick	Graph-Ansicht (Kür)

Deine Aufgabe (10 Minuten): drei bis fünf eigene Punkte, jeder so konkret, dass du am Ende prüfen kannst, ob er erfüllt ist.

VORAUSSETZUNGEN

- Deine Notizen als Markdown-Dateien (z. B. ein Obsidian-Vault) — sie bleiben, wo sie sind: keine Migration
- Claude Code installiert (claude.com/claude-code)
- Etwa eine Stunde Zeit für Stufe 1 — sie liefert dir den größten Teil des praktischen Nutzens

Stufe 1 — Das Fundament in einer Stunde

Kein Code, keine App. Nur Ordnung, ein Katalog, eine Suche und ein Regelwerk.

1 Datenbasis ordnen — mit Claude Code

Die beste Suche findet in einem Chaos-Vault nur Chaos schneller. Lass Claude deinen Bestand durchsuchen und ordne gemeinsam mit ihm neu: wenige Themen-Cluster plus Workflow-Ordner (Inbox, Daily Notes, Vorlagen, Archiv). Claude schlägt vor — du entscheidest.

PROMPT-VORLAGE (IN CLAUDE CODE, IM NOTIZ-ORDNER GESTARTET)

Durchsuche meinen Notiz-Ordner und schlage mir eine Cluster-Struktur vor: wenige Themen-Cluster plus Workflow-Ordner (Inbox, Daily Notes, Vorlagen, Archiv). Zeige den Vorschlag als Tabelle (Notiz zu Ziel-Ordner) und verschiebe erst nach meiner Freigabe – Schritt für Schritt, nicht alles auf einmal.

2 INDEX.md anlegen — der Katalog

Eine einzige kleine Datei, die dein ganzes System in Kurzform beschreibt. Sie ist die erste Stufe der Suchleiter und so kurz, dass sie immer komplett gelesen werden kann.

PROMPT-VORLAGE

Lege im Wurzelordner eine INDEX.md an: ein Katalog mit einer Zeile pro Bereich – Name, Pfad, Ein-Satz-Beschreibung. Maximal wenige hundert Zeilen. Ergänze die Regel: Jeder neue Bereich bekommt sofort eine Indexzeile.

3 qmd installieren — die Bedeutungssuche

qmd (von Shopify-Gründer Tobi Lütke) kombiniert Stichwortsuche mit Bedeutungssuche. Drei kleine Spezialmodelle werden bei der Einrichtung automatisch geladen und laufen komplett lokal — keine Cloud, keine Kosten, funktioniert offline.

TERMINAL (INSTALLATION: SIEHE README AUF [GITHUB.COM/TOBI/QMD](https://github.com/tobi/qmd))

```
# Notiz-Ordner als Sammlung registrieren, dann indexieren:
qmd collection add ~/Pfad/zu/deinen/Notizen
qmd embed

# Testen – absichtlich unscharf formulieren:
qmd query "Wie war unser Stil für die Untertitel?" -n 5

# An Claude Code anbinden:
claude plugin install qmd@qmd
```

4

Brain-First-Regeln in die CLAUDE.md — das Regelwerk

Das Entscheidende ist nicht die KI, sondern ihr Regelwerk. Diese Suchleiter sorgt dafür, dass Claude nie blind sucht und nie den halben Vault einliest — pro Frage landen nur wenige tausend Token im Kontext, egal wie groß dein Bestand wird.

COPY-PASTE IN DEINE CLAUDE.MD

```
## Brain-First-Protokoll (Suchleiter)
```

Bei jeder Wissensfrage gilt diese Reihenfolge:

1. Lies zuerst INDEX.md (den Katalog) – steht die Quelle dort, öffne genau diese.
2. Prüfe dann das Wiki (verdichtetes Wissen), falls vorhanden.
3. Erst danach die Suche (qmd query) – Kandidaten prüfen, ohne Dateien zu öffnen.
4. Öffne genau EINE Datei – die beste – und lies nur die relevante Sektion.
5. Dann erst antworten. Kein blindes Durchsuchen ganzer Ordner.

5

Testen — der Praxisvergleich

Was du nicht testest, kannst du dir leicht schönreden. Der einfache Test: dieselbe Frage in zwei frischen Claude-Code-Sessions — einmal ohne dein System (in einem neutralen Ordner), einmal mit. Nach der Antwort jeweils **/cost** eintippen und Token plus Zeit vergleichen. Mit **/context** siehst du den Füllstand des Kontextfensters als Balken.

Ehrliche Erwartung: Bei einfachen Fragen ist der Unterschied klein. Das System gewinnt bei tief vergrabenem Wissen und Fragen über mehrere Dateien — bei mir: minus 50 Prozent Token, minus 40 Prozent Zeit über fünf echte Alltagsfragen.

MEILENSTEIN

Damit steht das Fundament: **Ordnung, Katalog, Bedeutungssuche, Regelwerk, Praxistest**. Alles Weitere ist Ausbau — in deinem Tempo.

Stufe 2 — Das Wiki: Wissen, dem du vertrauen kannst

Das Muster stammt von KI-Forscher Andrej Karpathy — kein Programm, nur eine Idee: **Die KI pflegt ein eigenes Wissens-Wiki nach festen Regeln**. Sie verdichtet Quellen zu Seiten, vernetzt sie und markiert Widersprüche, statt sie stillschweigend aufzulösen. Der Wiki-Ordner ist ganz normales Obsidian — Obsidian ist das Regal, Claude ist der Bibliothekar.

1

Wiki-Ordner und Regel-Datei anlegen

COPY-PASTE ALS 09 WIKI/_SCHEMA.MD

Regeln dieses Wiki

1. Nur die KI schreibt hier. Ich lese, frage und kuratiere Quellen.
2. Rohquellen sind unantastbar – das Wiki liest sie, ändert sie nie.
3. Jede Seite ist vernetzt: mindestens ein Link auf eine andere Seite.
4. Widersprüche werden MARKIERT, nie aufgelöst: beide Aussagen bleiben stehen, mit Warnhinweis und beiden Quellen. Ich entscheide.
5. Jede Wiki-Änderung endet mit einem Eintrag in log.md (Datum, Quelle, geänderte Seiten).
6. Seitentypen: Quellen (eine Seite pro Quelle), Themen (verdichtet), Entitäten (Steckbriefe), Synthesen (These + Belege + offene Fragen).

2

Quellen einarbeiten lassen (Ingest)

PROMPT-VORLAGE (TERMINAL, CLAUDE-CODE-APP ODER CLAUDIAN IN OBSIDIAN)

Arbeite diese Quelle in mein Wiki ein: [Pfad oder Link].

Halte dich an die Regeln in 09 Wiki/_SCHEMA.md.

Gleiche mit dem bestehenden Wiki ab: Widersprüche markieren, nie stillschweigend auflösen. Schreibe zum Schluss den Log-Eintrag.

Bei mir hat der erste Bestands-Durchlauf 52 Konfliktstellen in meinem eigenen Wissen markiert — echte Widersprüche, veraltete Stände, unklare Soll-Ist-Trennungen. Sichtbar wurden sie erst durch die Verdichtung.

3

Der Widerspruchs-Workflow: Was tun mit einem Fund?

Der Warnhinweis ist kein Endzustand — er ist eine Aufgabe. Die Reihenfolge ist entscheidend:

1. **Entscheiden, was gilt.** Drei mögliche Urteile: der alte Stand ist veraltet / beides stimmt, aber der Kontext fehlt (z. B. Soll gegen Ist) / die neue Information ist falsch. Auch legitim: bewusst offen lassen.
2. **Die QUELLE reparieren — nicht den Warnhinweis.** Löscht du nur den Hinweis, findet der nächste Durchlauf denselben Konflikt wieder, denn die Dateien widersprechen sich weiterhin.
3. **Das Wiki nachziehen lassen:** „Der Konflikt ist entschieden, es gilt X, die Quelle ist angepasst — aktualisiere das Wiki.“ Die Entscheidung landet im Protokoll.

Manchmal ist die Entscheidung: Das Ziel bleibt, die Zahlen müssen besser werden. Dann wird aus einem versteckten Widerspruch ein sichtbarer Arbeitsauftrag.

Stufe 3 — Die Optik: Bilder statt Beschreibungen

Erst wenn das System steht und sich bewiesen hat, kommt das Fenster. Und hier der wichtigste Rat aus eigener, teurer Erfahrung:

WAS DU NICHT MACHEN SOLLTEST

In Text beschreiben, wie es aussehen soll, und die KI bauen lassen. „Schöner“, „dichter“, „mehr Tiefe“ — das ist für eine KI kein Auftrag, sondern ein Ratespiel. Das kostet Stunden und endet in Frust.

Stattdessen: Lass dir von einer Bild-KI (z. B. ChatGPT) zuerst den Zielzustand als Bild generieren. Beschreibe dein Problem, hole dir ein Konzept, iteriere an den Bildern — und gib die fertigen Mockups dann Claude als Referenz-Input. Für Inspiration lohnt auch Pinterest (Suchbegriffe wie „Knowledge Graph“ oder „Big Data Cloud“).

PROMPT-BEISPIEL FÜR DAS ZIELBILD (BILD-KI)

Ich benötige Ideen für eine Graphenansicht für mein Second Brain. Mein Wissen ist in bis zu 10 Hauptcluster unterteilt. Jedes Cluster soll eine eigene Farbe haben und separat sein, sodass sich die Inhalte nicht mischen. Optisch hochwertiger als der Obsidian-Graph. Erstelle mir zuerst ein Konzept, dann ein Bild davon.

Dann die Bau-Regel: **ein Schritt, Abnahme im Browser, nächster Schritt**. Nie alles auf einmal. Mit Bildern als Referenz sitzt jede Ansicht in Minuten — Geschmack kannst du nicht delegieren, aber zeigen.

WENN DU DIE KÜR BAUEN WILLST — DER STARTPUNKT

Schreib mir ein Skript ohne KI, das durch alle meine Notizen geht, Titel, Links und Tags einsammelt und daraus zwei Dateien schreibt: eine Landkarten-Datei (graph.json) und einen Katalog (INDEX.md). Baue danach eine lokale Web-Ansicht für graph.json — mit einer fertigen Graph-Bibliothek als Baustein (z. B. sigma.js oder d3-force). Visuelles Soll sind die angehängten Referenzbilder. Ein Schritt, dann Abnahme im Browser, dann der nächste.

So sind alle drei Rollen besetzt: Baustein (fertige Bibliothek), Muster (dieser Prompt), Messlatte (deine Mockups als Anhang). Nie als Ein-Satz-Auftrag — ohne Baustein und Messlatte wird das Ergebnis funktional, aber karg.

Links

qmd — lokale Hybridsuche	github.com/tobi/qmd
Karpathy — LLM Wiki (das Muster)	gist.github.com/karpathy/442a6bf555914893e9891c11519de94f
Superpowers — Workflow-Skills	github.com/obra/superpowers
Claude Code	claude.com/claude-code

Der Bau-Prozess für alles Größere: 6 Schritte

Egal ob Indexer, Wiki oder App — jedes größere Teil entsteht nach demselben Muster. Die Schritte 2 bis 5 übernimmt das Skill-Paket **Superpowers** für Claude Code: Es zwingt die KI in einen Ingenieurs-Prozess, und an jedem Übergang entscheidest du.

Schritt	Was passiert
1 · Datenbasis ordnen	Vault in Cluster-Struktur bringen — vor allem anderen
2 · Brainstorming	Die KI stellt Fragen, eine nach der anderen — du entscheidest. Noch kein Code
3 · Spec	Alle Entscheidungen schriftlich — das Dokument ist der Vertrag
4 · Plan	In kleine Aufgaben zerlegt, jede mit Test und Fertig-Kriterium
5 · Bauen	Aufgabe für Aufgabe, Prüfung dazwischen — du liest Ergebnisse, keinen Code
6 · Testen	Praxisvergleich gegen die normale Session plus Bewährung im Alltag

Die 7 Lektionen aus dem Projekt

- ☐ **Referenzen schlagen Beschreibungen** — bei Technik wie bei Optik. Drei Rollen: Baustein (fertig übernehmen), Muster (Idee adaptieren), Messlatte (Zielbild selbst erzeugen)
- ☐ **Kleine Schritte mit Abnahme** — nie alles auf einmal
- ☐ **Entscheiden vor Bauen** — die Spec ist der Vertrag
- ☐ **Deterministisch, wo es geht** — KI nur, wo Verstehen nötig ist; alles andere macht normaler Code kostenlos und wiederholbar
- ☐ **Markdown bleibt die einzige Wahrheit** — alles andere ist eine abgeleitete Sicht und jederzeit ersetzbar
- ☐ **Jede Behauptung braucht einen Prüfpunkt** — sonst glaubst du nur, statt zu wissen
- ☐ **Der Mensch entscheidet — die KI liefert** — bei jeder Frage, jedem Widerspruch, jedem Optik-Schritt

DER KERNSATZ

Die KI schreibt den Code. **Du** kuratierst die Referenzen, entscheidest — und nimmst jeden Schritt ab.